CENG110 - Programming and Computation II

Project 6 - Simple Banking System Report

- **240208435 - YUSUF TUNA GÖKER – System Coordination**
- **240208436 - MUSTAFA EBUBEKİR KAKİLLİ – Testing**
- **240208450 - ADA İNCAMAN – Documentation**
- **240208425 - YİĞİT ARDA KIDIMAN – ADT Design**

1. Introduction

The code is split into four modules. “main.py” handles the console interface. “bank.py” manages all accounts. “account.py” represents individual accounts. “transaction.py” tracks transactions using a stack.

Data is hidden through encapsulation. Users see a menu; the actual accounts, balances, and transaction histories stay behind the scenes through private attributes.

2. Abstract Data Type Design

The project uses two main ADTs: Account and Bank. It also includes Transaction and TransactionStack to handle transaction history.

2.1 Account ADT

The Account ADT stores one bank account with its number, holder name, balance, and transaction history.

An Account can:

- Store and return account information (number, name, balance)

- Handle deposits and withdrawals

- Send and receive money during transfers

- Keep transaction records

The Account uses private attributes like “__account_number”, “__holder_name”, and “__balance”. Other code can't reach these directly—they can only go through public methods like “deposit()” or “withdraw()”.

This matters because bank data shouldn't be touched from the outside. The “deposit()” method checks that amounts are positive. The “withdraw()” method checks both that the amount is positive and that the account has enough money. These guards protect the system.

2.2 Bank ADT

The Bank ADT is the whole system. It holds all accounts and runs system-level operations: create accounts, find accounts, deposit, withdraw, transfer, list accounts, search by name, and sort by balance.

The Bank stores accounts in a private dictionary "__accounts", where account numbers are keys. It also tracks "__next_account_number" starting from 1001.

The Bank handles:

- Creating new accounts with unique numbers
- Rejecting duplicate names
- Looking up accounts
- Checking if an account exists
- Deposits, withdrawals, transfers
- Returning balances and transaction histories
- Listing all accounts
- Sorting by balance
- Calculating total bank balance
- Searching by name

The Bank separates system-level decisions from account-level ones. The Bank checks that both account numbers are valid before a transfer; the Account handles how balance gets updated. This split keeps things clear.

2.3 Transaction and TransactionStack ADTs

The Transaction ADT represents one banking operation: timestamp, type, amount, new balance, and description. Types include initial deposit, deposit, withdraw, transfer out, and transfer in.

The TransactionStack stores transactions using a stack (last in, first out). Recent transactions sit at the top, which is what users usually want to see.

TransactionStack supports:

- "push": Add a transaction
- "pop": Remove the most recent one
- "peek": View the top without removing
- "isEmpty": Check if empty
- "size": Count transactions
- "get_transaction_history": Return all transactions in recent-first order
- "clear_stack": Empty the stack

3. Data Structure Justification

The project uses a stack inside TransactionStack, built from a Python list.

A stack works on LIFO—last in, first out. The newest transaction goes on top. When users look at their history, they expect recent activity first. That's what a stack gives them.

Stacks are also efficient for this. Each new transaction becomes a Transaction object and gets pushed on. Python's list “append()” is O(1), so recording new transactions is fast.

For account balance sorting, the program uses Bubble Sort. It's not the fastest, but it's simple and good enough for learning. It also lets us analyze sorting, which the assignment needs.

4. Algorithmic Analysis

This section looks at the time complexity of the key operations: add, remove, search, and sort.

4.1 Add Operation

There are two add operations: adding an account to the bank and adding a transaction to the stack.

Adding a transaction to TransactionStack uses list “append()”. That's O(1) on average because the item goes to the end.

Creating a new account is mostly O(1) for inserting the Account object into the dictionary. But the current code also checks for duplicate names by scanning all existing accounts. With N accounts, that check is O(N). So the full create operation is O(N).

4.2 Remove Operation

The stack remove operation is “pop()”, which takes the last transaction. Since we're removing the last list element, that's O(1).

The banking system doesn't have a menu option to delete accounts yet. So the main remove operation we analyze is removing a transaction from the stack.

4.3 Search Operation

Searching for an account by number uses the bank dictionary. Since numbers are keys, “get()” is O(1) on average.

Searching for an account by name requires checking every account because names aren't keys. With N accounts, that's O(N).

4.4 Sorting Operation

The program sorts accounts by balance using Bubble Sort. It swaps neighboring elements if they're in the wrong order. Here, accounts are sorted highest-balance-first.

With N accounts, Bubble Sort runs two nested loops. In the worst and average case, that's $O(N^2)$. Space is O(N) because we create a list of accounts from the dictionary, then sort it in place without building extra large structures.

Bubble Sort works for this project because we're learning and analyzing sorting. For a real-world bank with many accounts, merge sort, quicksort, or Python's built-in Timsort would be better.

5. Testing Plan and Implementation

The project includes a comprehensive automated test suite in `test_banking.py`. The following test cases cover normal operations, edge cases, invalid inputs, and complex scenarios.

5.1 Test Coverage Details

Test 1: Account Creation (Normal Case)

- Creates an account with holder name "Ahmet Yilmaz" and initial balance 1000
- Verifies account number is assigned (1001)
- Verifies holder name and balance are stored correctly

Test 2: Empty System (Edge Case)

- Verifies empty bank has 0 accounts
- Verifies total balance is 0 in empty system
- Verifies querying non-existent account returns None

Test 3: Invalid Deposit (Invalid Input)

- Attempts to deposit negative amount (-100)
- Verifies rejection of negative deposit
- Confirms balance unchanged after failed operation

Test 4: Insufficient Balance (Edge Case)

- Attempts to withdraw 500₺ from account with 300₺
- Verifies withdrawal is rejected
- Confirms balance remains 300₺

Test 5: Multiple Operations Sequence (Complex Scenario)

- Creates two accounts: "Ali Demir" (1000₺) and "Fatma Sultan" (500₺)
- Deposits 500₺ into first account → balance becomes 1500₺
- Transfers 300₺ from first to second account
- Verifies: First account = 1200₺, Second account = 800₺
- Withdraws 100₺ from second account → balance becomes 700₺
- Verifies total bank balance = 1900₺

Test 6: Duplicate Account Name (Duplicate Record Scenario)

- Creates first account "Ismail Yilmaz" with 1000₺
- Attempts to create second account with same name (case-insensitive: "ismail yilmaz")
- Verifies second account creation is rejected (returns None)
- Confirms only one account exists in the bank
- Tests duplicate prevention feature regardless of letter case

Test 7: Transaction History Stack (Stack LIFO)

- Creates account "Ilhan Mansiz" with 100₺
- Performs multiple operations: deposit 50₺, deposit 25₺, withdraw 30₺
- Verifies all transactions recorded (at least 4: initial, 3 operations)
- Confirms last transaction is a withdrawal (LIFO principle)

Test 8: Algorithm Complexity Analysis

- Documents all operation complexities:
- Create Account: O(N) - Includes duplicate name check
- Deposit/Withdraw: O(1)
- Transfer: O(1)

6. Conclusion

The Simple Banking System is a modular console application using ADTs and encapsulation. The Account ADT handles individual accounts; the Bank ADT manages all accounts and coordinates operations. The TransactionStack uses a stack to store transaction history, newest first.

The system supports the core requirements: create accounts, deposit, withdraw, transfer, and view history. It also has a sorting operation that lists accounts by balance using Bubble Sort. The Big-O analysis covers the main add, remove, search, and sort operations.